This assessment contains four progressively backend tasks. The goal is to evaluate API fundamentals, database knowledge, business logic, transaction handling, performance, concurrency, and code quality.

Note: Use LinQ, Create a Database on your local.Database name-> RetailDB. For all tasks make one web api solution. And use repository pattern (Controller-> Interface-> Repository) . Maintain SOLID and also try to use common functions for reuse.

General Requirements

- ASP.NET Core Web API (.NET 8 preferred; .NET 6+ acceptable).
- SQL Server with Entity Framework Core.
- Use DTOs instead of exposing EF entities directly from API responses.
- Use proper HTTP status codes and meaningful error responses.
- Use async/await for database operations.
- Provide database migration or SQL script.
- Use Git and provide a README with setup and API instructions.
- **Do databases first. Not code first.**

Task 1 — Product Management API

Build a basic Product Management REST API to demonstrate CRUD, validation, DTOs, EF Core, and HTTP status code knowledge.

Entity

```
Product
-------
Id
Name
Code
Price
Stock
IsActive
CreatedAt
```

Required APIs

- POST /api/products
- GET /api/products
- GET /api/products/{id}
- PUT /api/products/{id}
- DELETE /api/products/{id}

Requirements

- Product Code must be unique.
- Name is required.
- Price must be greater than 0.
- Stock cannot be negative.
- Deleted products should preferably be handled using IsActive/soft delete.
- Return 400 for invalid input, 404 when the product is not found, and appropriate success status codes.
- Use request/response DTOs.

- Pagination using pageNumber and pageSize.
- Search by product name/code.
- Sorting by created date desc.

Task 2 — Order Management & Inventory

Build an Order API where creating an order also validates product stock, calculates prices, and updates inventory.

Entities

```
Product
-------
Id
Name
Price
Stock

Order
-----
Id
CustomerName
OrderDate
TotalAmount

OrderItem
---------
Id
OrderId
ProductId
Quantity
UnitPrice
TotalPrice
```

Create Order API

```
POST /api/orders

{
  "customerName": "Rahim",
  "items": [
    { "productId": 1, "quantity": 2 },
    { "productId": 5, "quantity": 1 }
  ]
}
```

Requirements

- Validate that every product exists.
- Validate that requested quantity is greater than 0.
- Validate sufficient stock for every product.
- UnitPrice must come from the database, not from the client request.
- Calculate TotalPrice and TotalAmount on backend.
- Decrease product stock after successful order creation.
- If any validation fails, no order or stock change should remain.
- Use transaction in code label for order creation and stock update.

- Prevent duplicate ProductId entries within the same order.
- Provide GET /api/orders/{id} and GET /api/orders with basic filtering.
- Pagination and customer-name search.
- Order status: Pending, Confirmed, Cancelled.
- Cancel Order API that restores stock safely.

Task 3 — Warehouse Stock Transfer API

Build a warehouse-to-warehouse stock transfer module. The main focus is transactional business logic, validation, stock consistency, and concurrency.

Entities

```
Warehouse
---------
Id
Name
IsActive

Product
-------
Id
Name
Code
IsActive

WarehouseStock
--------------
Id
WarehouseId
ProductId
Quantity

StockTransfer
-------------
Id
FromWarehouseId
ToWarehouseId
TransferDate
Status

StockTransferItem
-----------------
Id
StockTransferId
ProductId
Quantity
```

Create Transfer API

```
POST /api/stock-transfers

{
  "fromWarehouseId": 1,
  "toWarehouseId": 2,
  "items": [
```

```
    { "productId": 10, "quantity": 5 },
    { "productId": 20, "quantity": 3 }
  ]
}
```

Requirements

- Source and destination warehouses must exist and be active.
- Source and destination warehouses cannot be the same.
- Every product must exist and be active.
- Every transfer quantity must be greater than 0.
- Duplicate ProductId entries in one transfer are not allowed.
- The source warehouse must have sufficient stock.
- Create transfer header and items.
- Decrease source warehouse stock and increase destination warehouse stock.
- All database changes must happen inside one transaction.
- If one item fails, the entire transfer must rollback.
- Do not allow stock to become negative.

Concurrency Requirement (Optional)

Assume two users try to transfer the same product from the same warehouse at almost the same time. Design the implementation so that both requests cannot consume the same stock.

The candidate should explain the chosen approach, such as:

- Database transaction/isolation level. (Optional)
- Atomic UPDATE with a quantity condition. (Optional)
- Row-level locking where appropriate. (Optional)
- Optimistic concurrency. (Optional)

Task 4 — High-Volume Sales Reporting & Performance API

Build a reporting API (First Do store procedure then call it in API and make the endpoint) that works efficiently with a large sales dataset.

Entities

```
Customer
--------
Id
Name
Phone

Product
-------
Id
Name
CategoryId
Price

SalesHeader
-----------
Id
```

```
CustomerId
SaleDate
InvoiceNo
TotalAmount
IsCancelled

SalesLine
---------
Id
SalesId
ProductId
Quantity
UnitPrice
DiscountAmount
LineTotal
```

Required API

```
GET /api/reports/sales-summary
GET /api/reports/product-sales
GET /api/reports/customer-sales
```

Sales Summary

Support filters such as:

- FromDate
- ToDate
- ProductId (optional)
- CustomerId (optional)
- CategoryId (optional)

Required Output

```
{
  "totalInvoices": 12540,
  "totalQuantity": 98450,
  "grossSales": 12500000,
  "discount": 450000,
  "netSales": 12050000
}
```

Product Sales Report

Return product-wise sales with total quantity, gross sales, discount, and net sales. The API must support pagination and sorting.

Performance Requirements

- The solution must be designed for large data.
- Do not load the entire dataset into application memory.
- Filtering and aggregation should happen in SQL.
- Avoid N+1 database queries.
- Use projection instead of loading unnecessary columns/entities.
- Pagination must be performed at the database level.
- **Use Dapper or optimized EF Core query for the reporting endpoint and explain the choice.**